

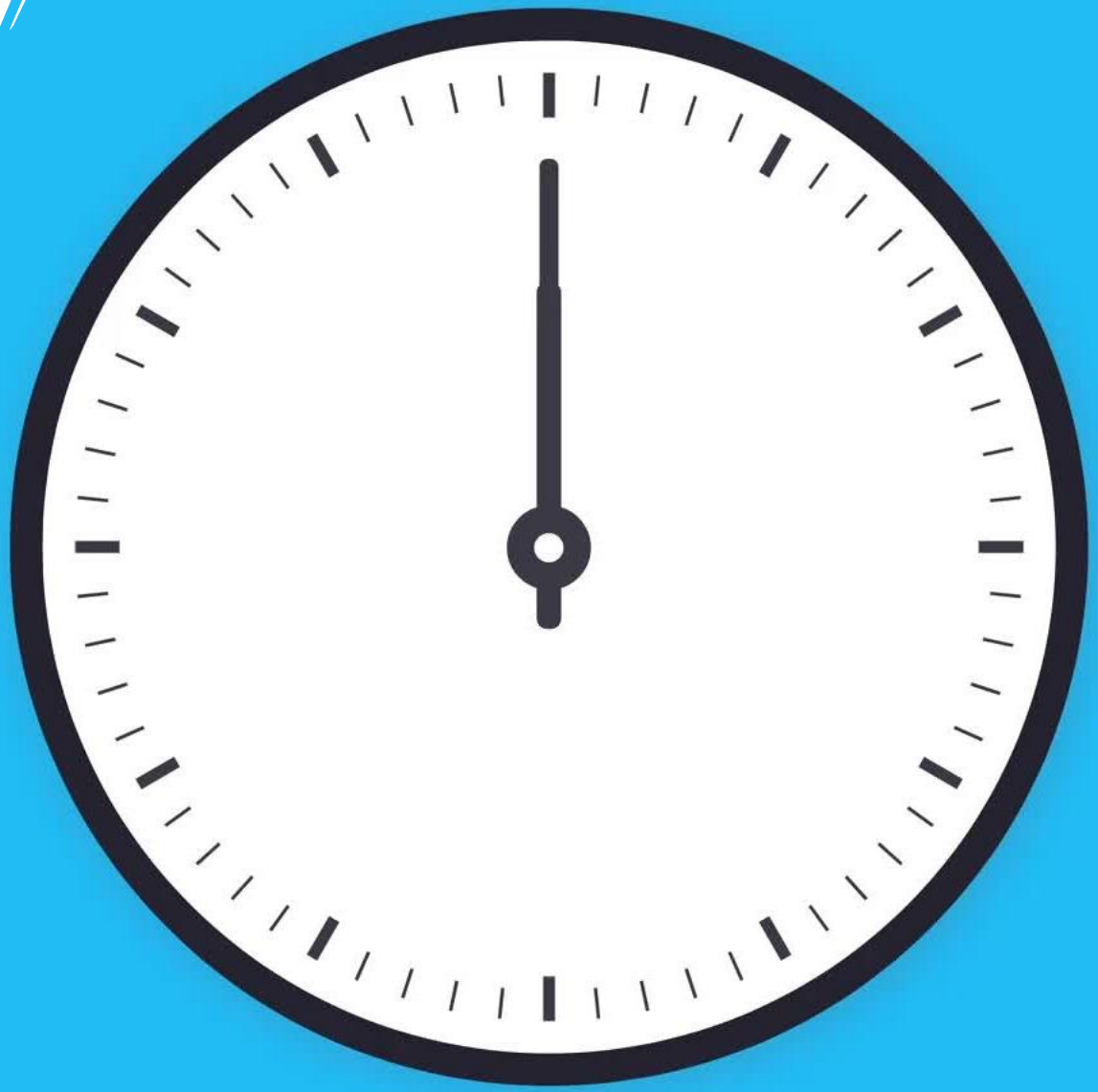
Linjär Algebra med fokus på 3D-matematik

Föreläsning 01

Vektorer, definition och aritmetik

Det vi kommer att göra idag

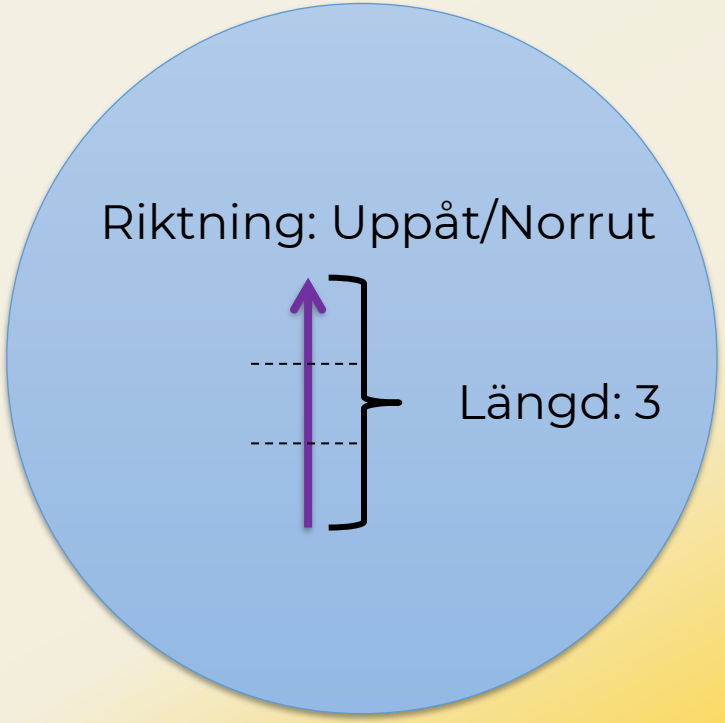
- Vad är en vektor?
  - Definition
- Att räkna med vektorer
  - Addition, Subtraktion
  - Multiplikation *(med skalär)*
  - *Division*
  - Enhetsvektor
- Lite flera operationer
  - *Normalisering*
  - *Vektormultiplikation*
    - *Komponentvis, skalärprodukt, kryssprodukt*
- Avslutningsvis



Vad är en vektor?

- En vektor ≠ std::vector
  - std::vector är en lista med objekt (*mutable array*)
  - En vektor är en matematisk vektor
  - Liknande men ej samma!

- Matematisk vektor
  - **Har** en riktning
  - **Har** en längd



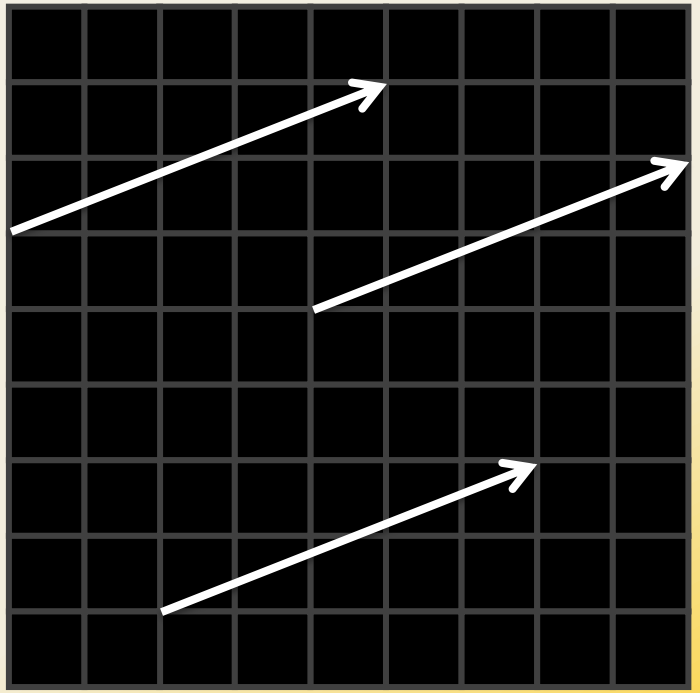
Vi kan rita ut en vektor

- En vektor definierar **inte en position**
- **Samma** vektor *”kan ritas ut på många ställen”*



Kan ritas ut i ett rutnät

Alla de tre **vektorerna** till höger är **samma** som **vektorn** till vänster

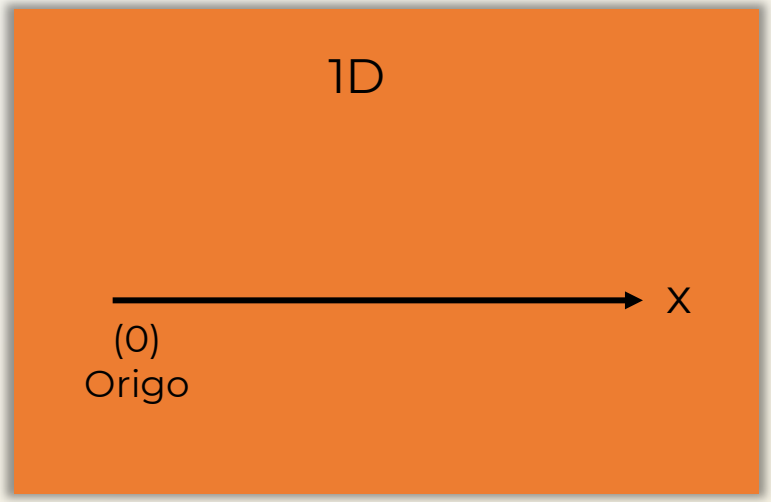


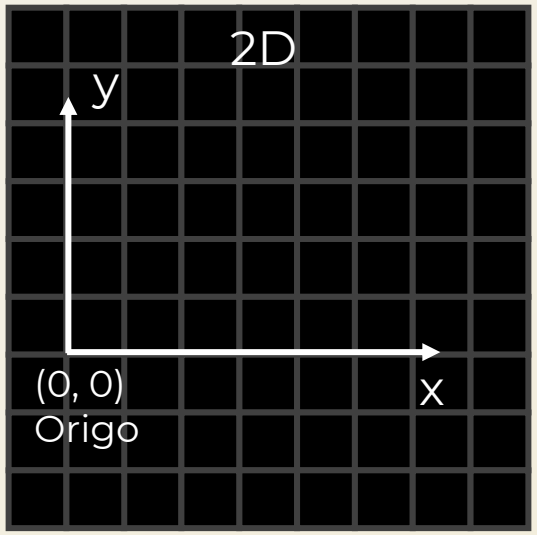
TGA
THE GAME ASSEMBLY

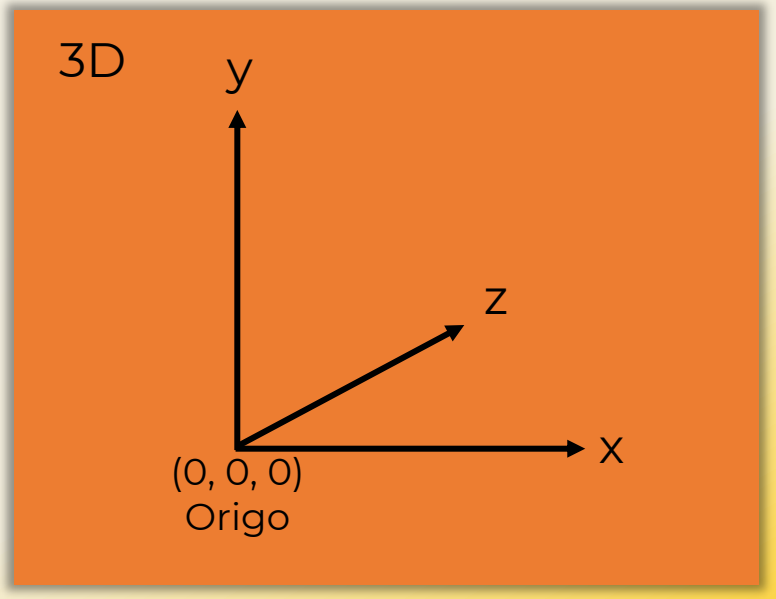
Ett koordinatsystem består av ett **origo** och **axlar**

Origo är nollpunkten (0, 0)

I **1 dimension** har vi **1 axel**, *en tallinje*
I **2 dimensioner** har vi **2 axlar**
I **3 dimensioner** har vi **3 axlar**
osv...

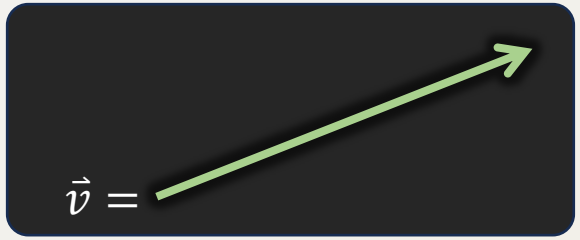




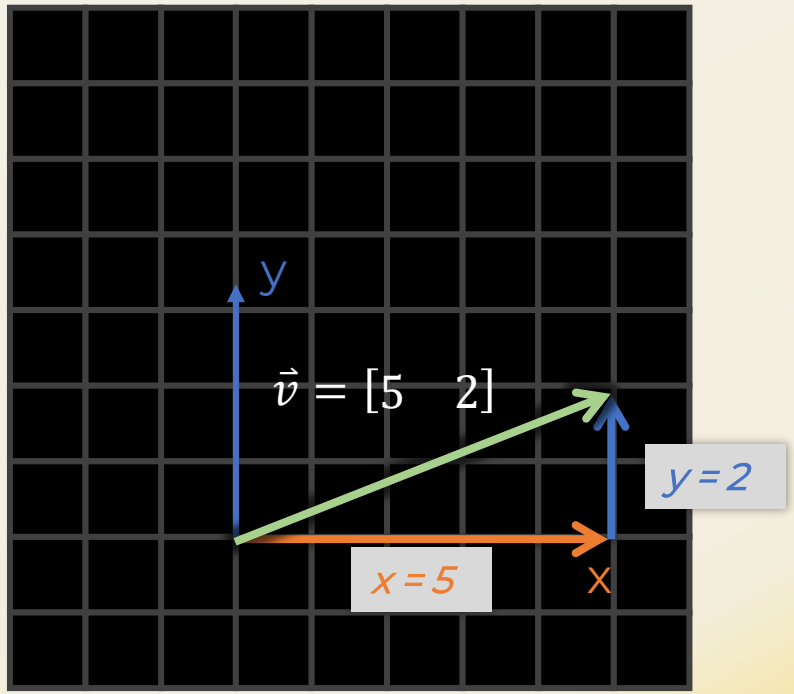


TGA
THE GAME ASSEMBLY

Definiera en vektor



1. Koordinatsystem
   - X-led (vänster till höger)
   - Y-led (nerifrån och upp)
2. Startar vektorn i *origo*
   - Där axlarna skär varandra
   - Punkt (0, 0)
3. Läs ut x och y från rutnätet
4. Vektorn har
   - x = 5
   - y = 2



Vi använder rad-vektorer som skrivs

$$\vec{v} = (v_x \quad v_y)$$

Komponenterna skrivs
v_x för x-koordinaten
v_y för y-koordinaten

Boken har kolumn-vektorer i **fetstil**

$$\mathbf{v} = \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

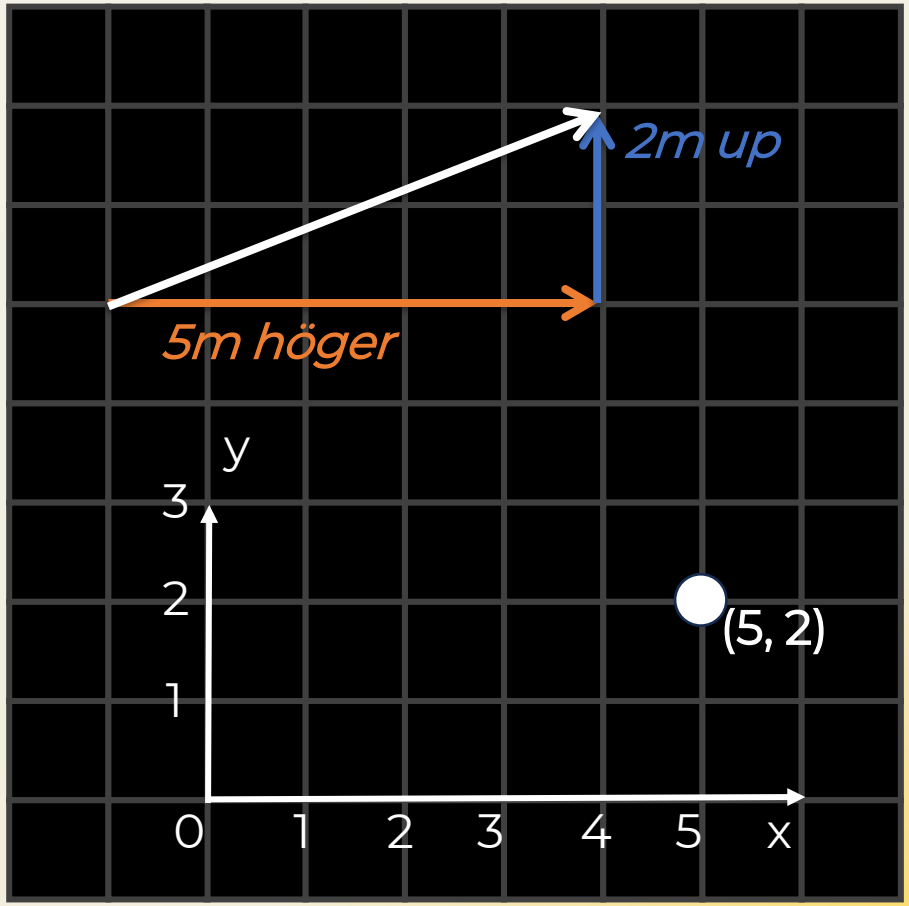
TGA THE GAME ASSEMBLY

Riktning och längd

- En vektor har **riktning** och **längd**
  - Definierar dock **inte position**

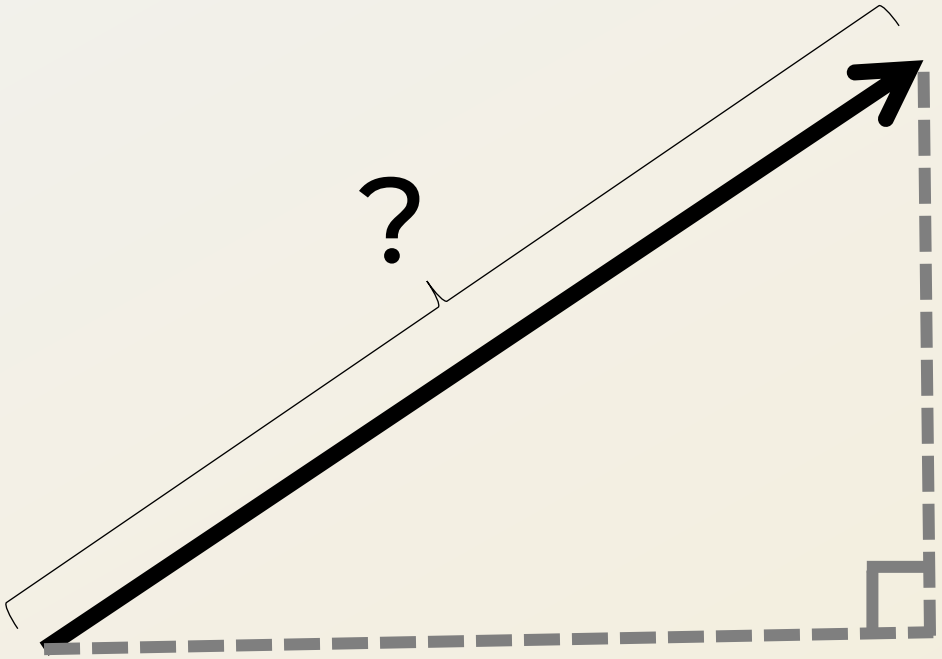
- **Men** vektorer *kan tolkas* som en position
  - Exempelvis för att beskriva en förflyttning
    - Från startpunkten
      - Gå **5m höger**
      - Gå **2m up**

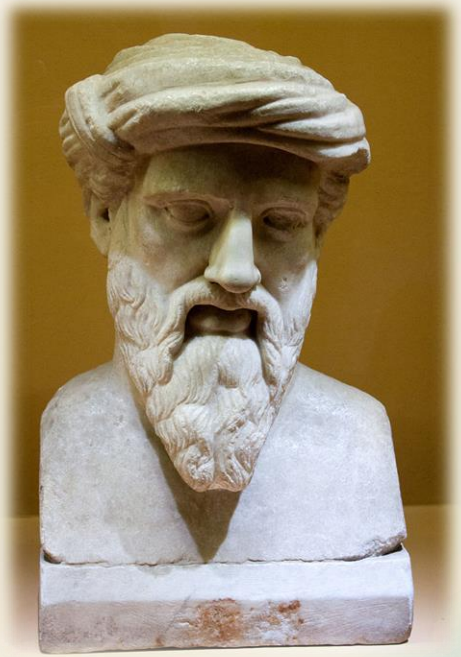
- Då kan den **tolkas** som en **koordinat** med en position
  - **Kallas** då ofta för en **punkt**, **koordinat** eller **position**



TGA
THE GAME ASSEMBLY

VEKTORERS LÄNGD





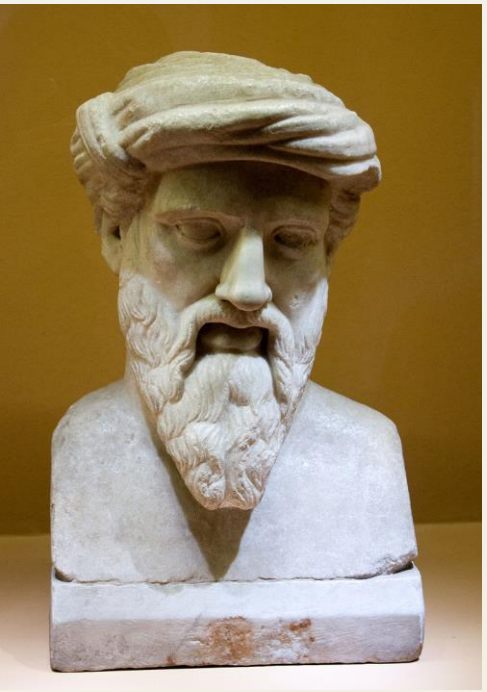
TGA
THE GAME ASSEMBLY

Vektorers längd

Längden av $\vec{v}$ skrivs såhär
$\|\vec{v}\|$

En 2D-**vektor** tillsammans med **axlarna** bildar en **rätvinklig triangel**

- Förändring längst axlarna utgör **kateterna** (v_x & v_y)
- Vektorn utgör **hypotenusan** $\vec{v}$



Vad säger Pytagoras?

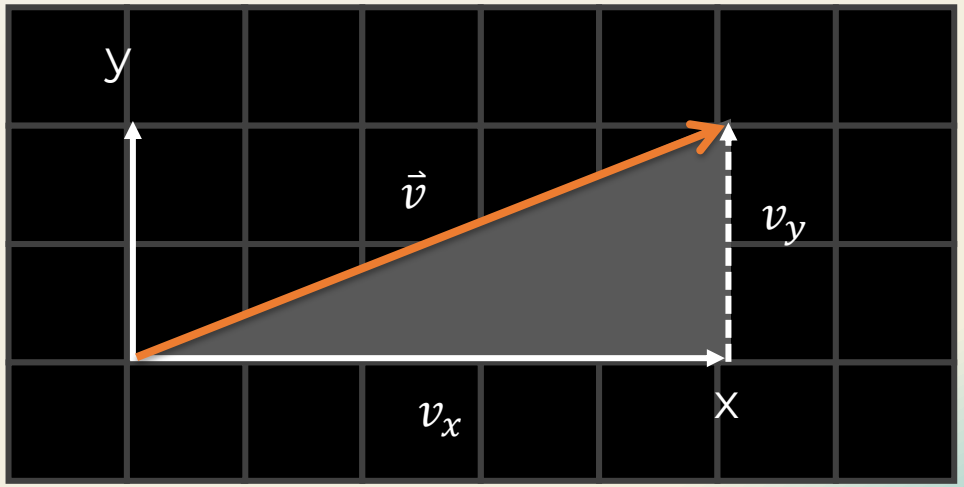
$$hypotenusa^2 = katet^2 + katet^2$$

Alltså längden av hypotenusan

$$hypotenusa = \sqrt{katet^2 + katet^2}$$

Vilket i exemplet blir

$$\|\vec{v}\| = \sqrt{{v_x}^2 + {v_y}^2}$$




```
float GetLength(float aX, float aY)
{
   return sqrt(aX*aX + aY*aY);
}
```

TGA
THE GAME ASSEMBLY

Avståndsformeln $$\boxed{\|\vec{v}\| = \sqrt{\vec{v}_x{}^2 + \vec{v}_y{}^2}}$$

Avståndsformeln **gäller** för vektorer **oavsett dimension**!

Vektor i 1 dimension: $\| \vec{v} \| = \sqrt{v_0{}^2}$

$\vec{v} = [3]$

$\| \vec{v} \| = \sqrt{3^2} = \sqrt{9} = 3$

Vektor i n dimensioner: $\| \vec{v} \| = \sqrt{v_0{}^2 + v_1{}^2 + \cdots + v_n{}^2}$

Vektor i 3 dimensioner: $\| \vec{v} \| = \sqrt{v_0{}^2 + v_1{}^2 + v_2{}^2}$

$= \sqrt{v_x{}^2 + v_y{}^2 + v_z{}^2}$

En **viktig** vektor är den med längden 1

$$\| \vec{v} \| = 1$$

Den kallas **enhetsvektor** och noteras oftast med en *hatt*

$\hat{v}$

TGA THE GAME ASSEMBLY

Vektorer, kodexempel

```
struct Vector2
{
   float x;
   float y;
};
```

```
struct Vector3
{
   float x;
   float y;
   float z;
};
```

```
struct Vector4
{
   float x;
   float y;
   float z;
   float w;
};
```

```
float Length(Vector2 aVec)
{
   return sqrt(aVec.x * aVec.x + aVec.y * aVec.y);
}

float Length(Vector3 aVec)
{
   return sqrt(aVec.x * aVec.x + aVec.y * aVec.y + aVec.z * aVec.z);
}
```

TGA
THE GAME ASSEMBLY

ATT RÄKNA MED VEKTORER

TGA
THE GAME ASSEMBLY

Vektormatematik

Vektorer kan ses som generaliserade tal

Ett vanligt tal (*skalär*) är en **1-dimensionell vektor**

Talet 0 representeras av *noll-vektorn* (*zero-vector* eller *null-vector*)

(0) (0, 0) (0, 0, 0) etc.

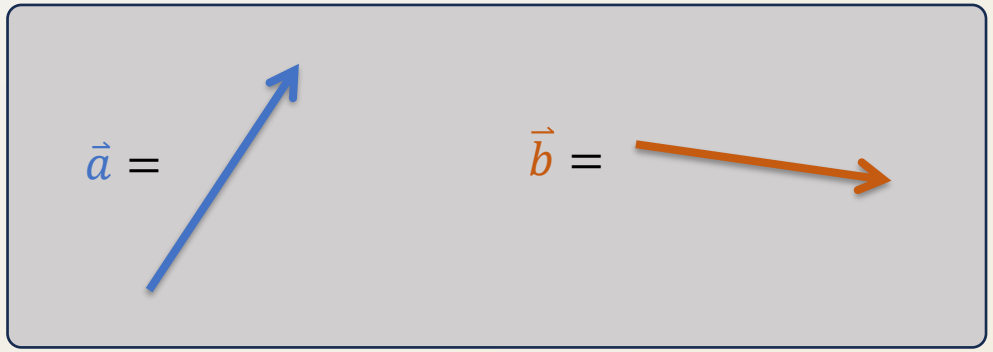
Origo i koordinatsystem
Har **längd 0**
Har **ingen** riktning

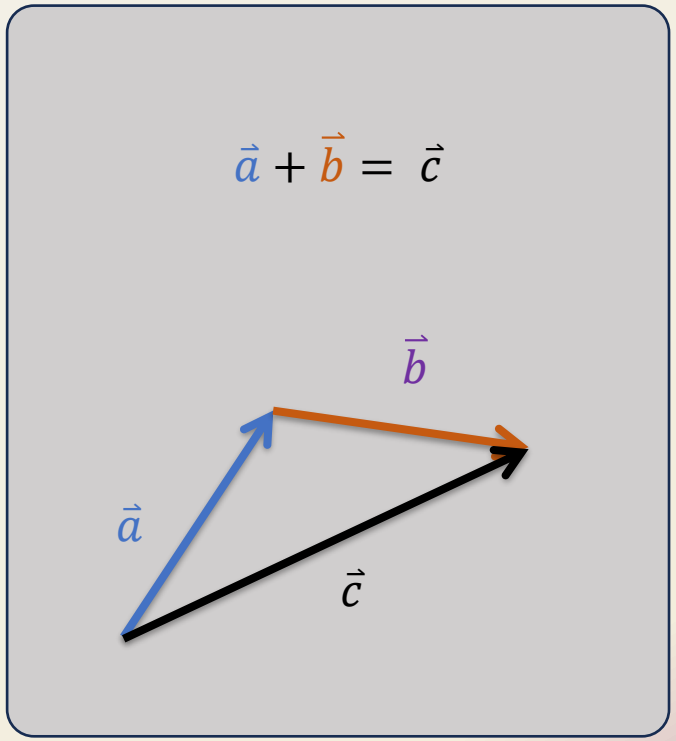
TGA
THE GAME ASSEMBLY

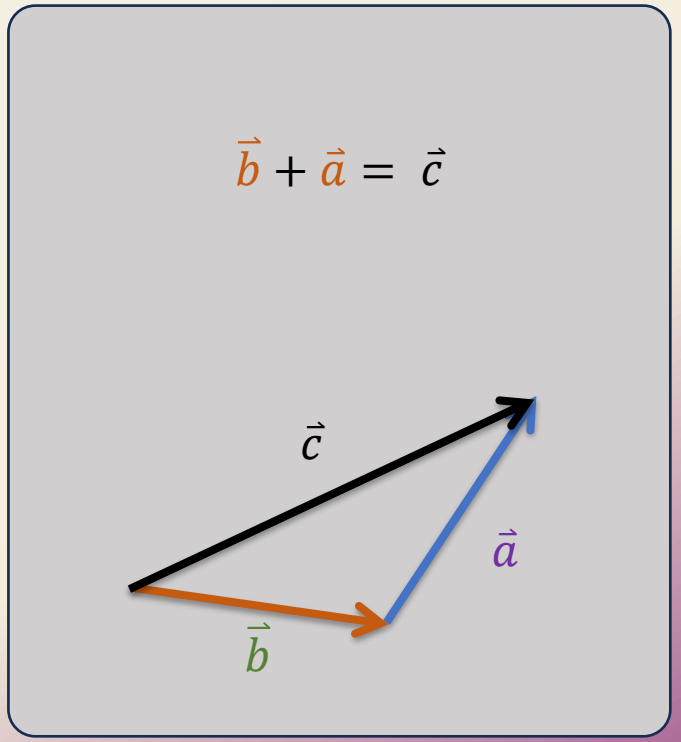
Addition – geometriskt

Addition är *bara* att “kombinera” pilarna

Ordningen spelar ingen roll







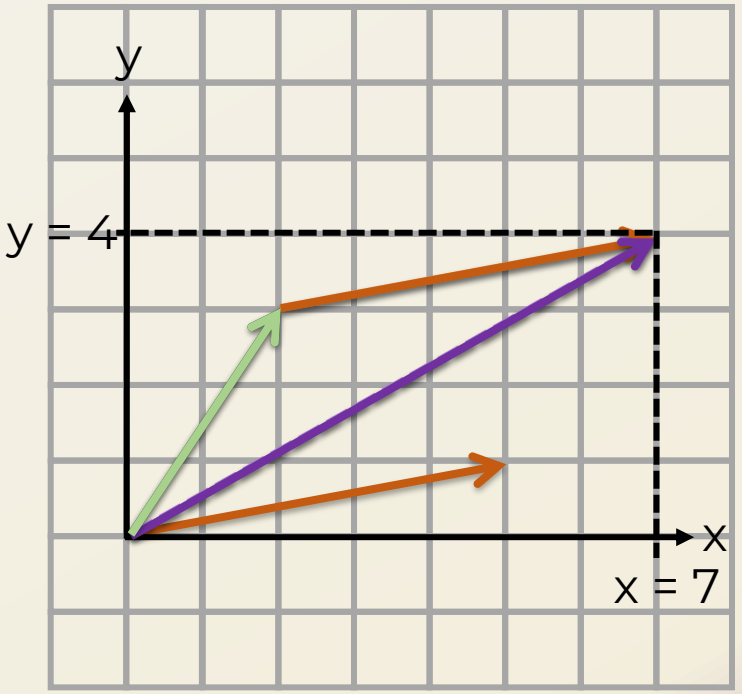
TGA
THE GAME ASSEMBLY

Addition – aritmetiskt

$$\vec{a} = (2, \quad 3)$$
$$\vec{b} = (5, \quad 1)$$

$$\vec{a} + \vec{b}(2, \quad 3) + (5, \quad 1)$$
$$= (2 + 5, \quad 3 + 1) = (7, \quad 4)$$

$$\vec{a} + \vec{b} = (a_x, a_y) + (b_x, b_y) = (a_x + b_x, a_y + b_y)$$



TGA
THE GAME ASSEMBLY

Addition av vektorer

Vektoraddition är **kommutativ**

Ordningen på operanderna **spelar ingen roll**

$$\vec{a} + \vec{b} = \vec{b} + \vec{a}$$

Addera komponentvist

$$(a_x + b_x, a_y + b_y) = (b_x + a_x, b_y + a_y)$$

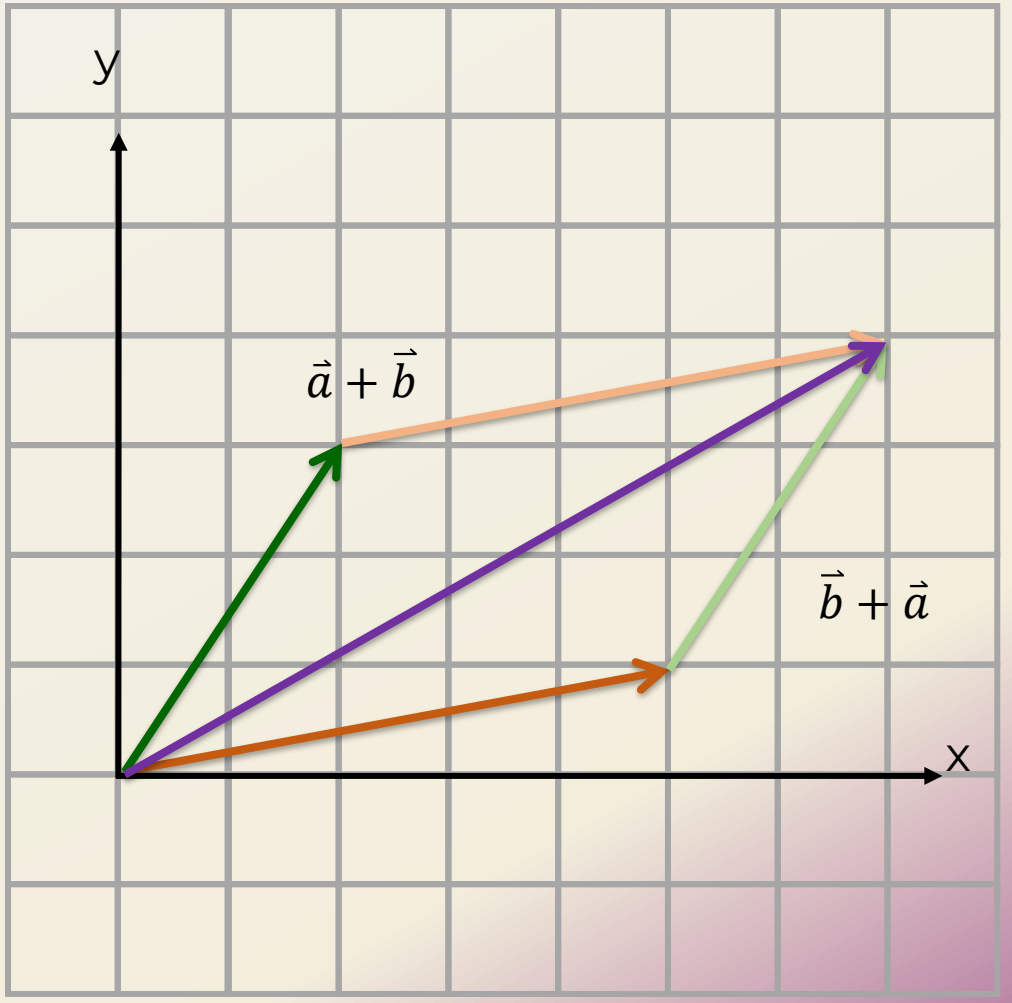
Vektoraddition är **associativ**

Ordningen på operationerna **spelar ingen roll**

$$(\vec{a} + \vec{b}) + \vec{c} = \vec{a} + (\vec{b} + \vec{c})$$

Vi kan addera

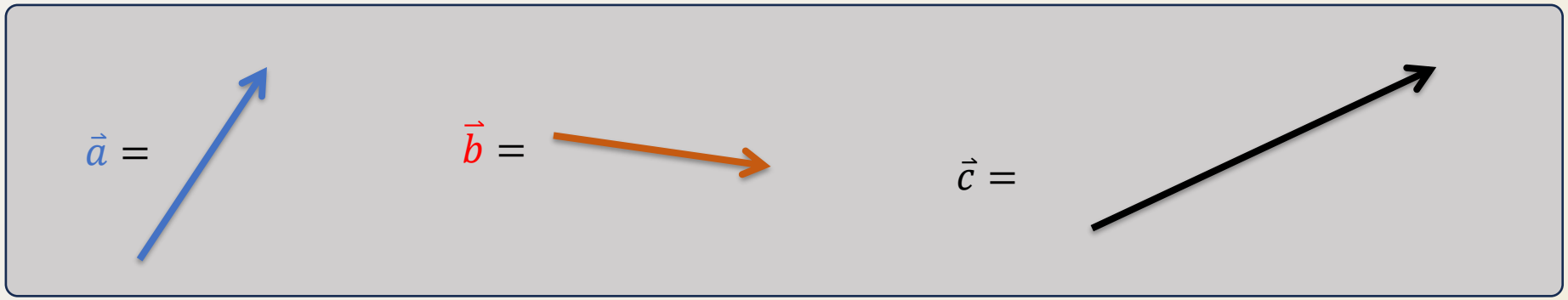
hur många vektorer som helst
i vilken ordning som helst
→ resultatet blir alltid detsamma



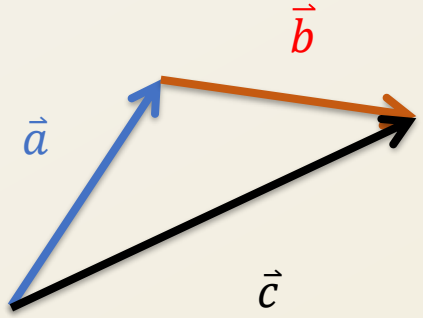
TGA
THE GAME ASSEMBLY

Subtraktion – geometriskt

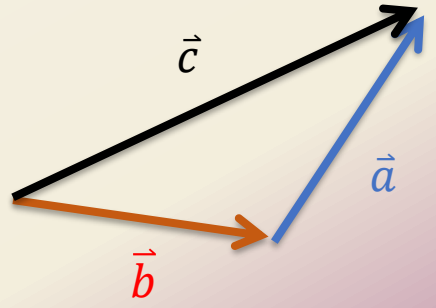
Subtraktion är *bara* att *gå baklänges* längs en vektor



$\vec{c} - \vec{b} = \vec{a}$



$\vec{c} - \vec{a} = \vec{b}$



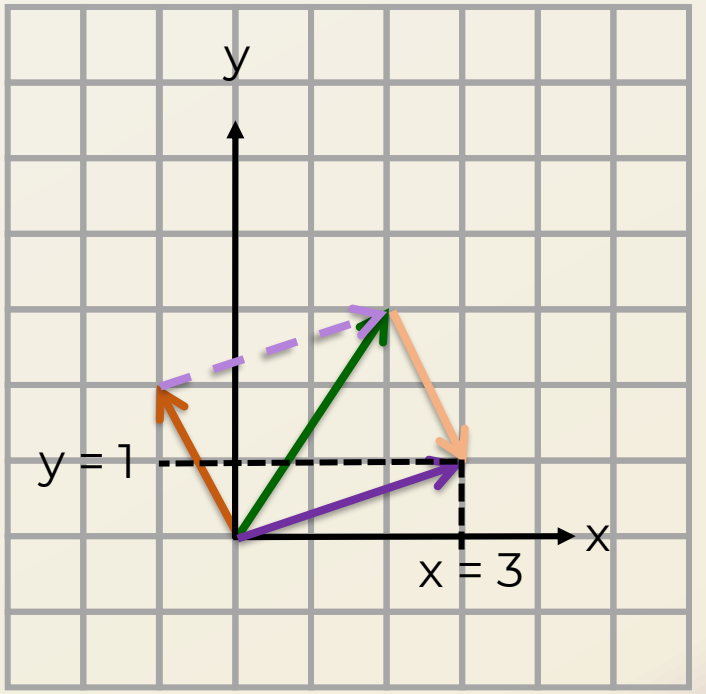
TGA
THE GAME ASSEMBLY

Subtraktion – aritmetiskt

$\vec{a} = (2, \quad 3)$
$\vec{b} = (-1, \quad 2)$

$\vec{a} - \vec{b}$:Vänd på $\vec{b}$ och addera till $\vec{a}$

$$\vec{a} + (-\vec{b}) = (2, \quad 3) + (-(-1, \quad 2))$$
$$= \big(2 + \big(-(-1)\big),\ 3 + (-2)\big)$$
$$= (2 + 1,\ 3 - 2)$$
$$= (3,\ 1)$$



$$\vec{a} - \vec{b} = (a_x, a_y) - (b_x, b_y) = (a_x - b_x,\ a_y - b_y)$$

Vi kan också se det som att vi drar en vektor från b till a

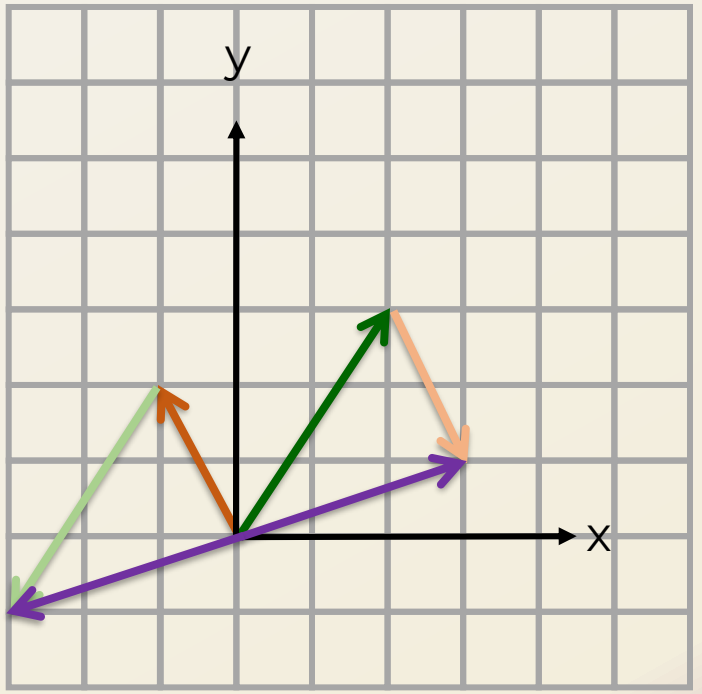
TGA THE GAME ASSEMBLY

Subtraktion av vektorer

Precis som vid subtraktion av skalärer gäller

$$\vec{a} - \vec{b} \neq \vec{b} - \vec{a}$$

$$\vec{a} - \vec{b}$$
$$\vec{b} - \vec{a}$$



Subtraktion av vektorer är **inte** kommutativ

Subtraktion av vektorer är **inte** associativ

TGA
THE GAME ASSEMBLY

Multiplikation av vektor och skalär

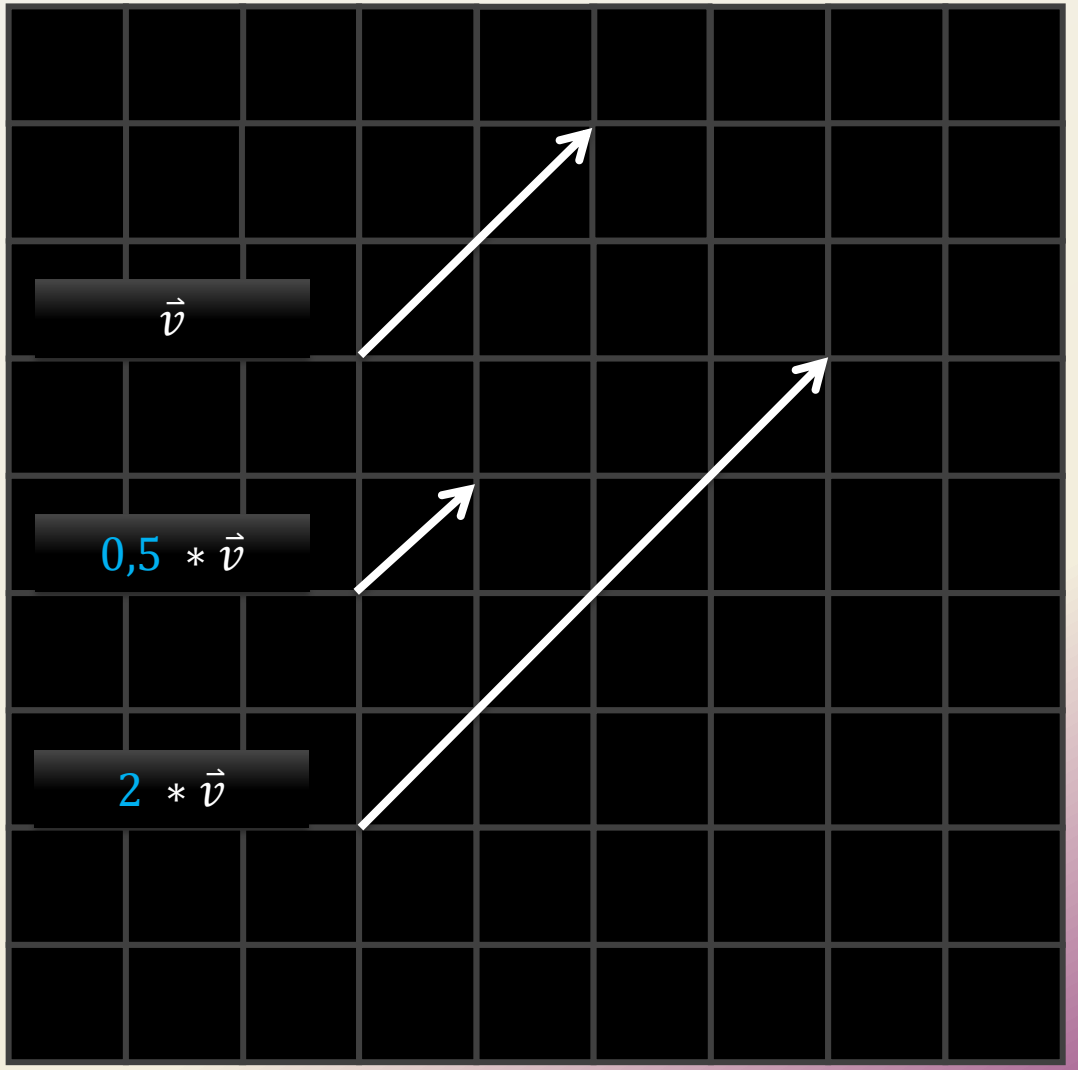
- Skalär = tal
- Heter ibland *skalärmultiplikation*
  - Jag föredrar ”*multiplikation med skalär*”

Skalar längden på vektorn

Behåller vektorns **riktning**

```
// Skalar hastigheten med dt
Vector2 movement = myVelocity * deltaTime;
```

$$s * \vec{v} = s(v_x, v_y) = (sv_x, sv_y)$$

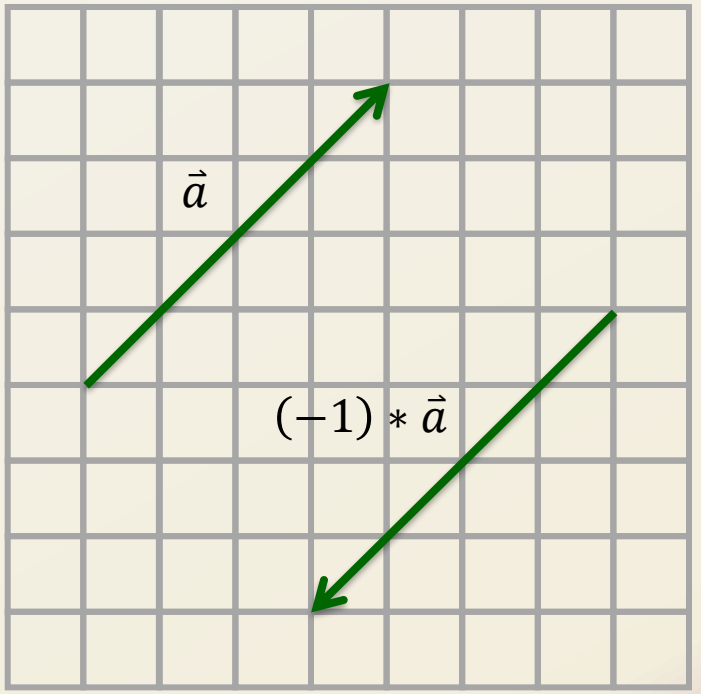


TGA
THE GAME ASSEMBLY

Multiplicera med skalär = skalar längden

Om vi skalar med (-1) är det detsamma som att **vända på vektorn**

- **Längden** är **densamma**
- **Riktningen** är **motsatt** – ”Pilen är i andra änden”



Detta använder vi i subtraktion:

$$\vec{a} - \vec{b} = \vec{a} + \left((-1) * \vec{b} \right)$$

TGA
THE GAME ASSEMBLY

Division - vektor med skalär

En division med s är detsamma som en multiplikation med $\frac{1}{s}$

$\frac{1}{s}$ Är en skalär. Vi kan multiplicera vektor med skalär

$$\vec{v} * s = (v_y * s \quad v_y * s)$$

$$\frac{\vec{v}}{s} = \vec{v} * \frac{1}{s} = (v_x * \frac{1}{s}, v_y * \frac{1}{s}) = (\frac{v_x}{s}, \frac{v_y}{s})$$

TGA
THE GAME ASSEMBLY

Sammanfattning

Addition	$\vec{a} + \vec{b} = (a_x + b_x, a_y + b_y)$
Skalärmultiplikation	$s * \vec{v} = (sv_x, sv_y)$
Skalärdivision	$\frac{\vec{v}}{s} = (\frac{v_x}{s}, \frac{v_y}{s})$
Vektorns längd	$\|\vec{v}\| = \sqrt{{v_x}^2 + {v_y}^2}$
Enhetsvektorer $\hat{e}$ har längden 1	$\|\vec{v}\| = 1$
Koordinataxlarna skrivs ofta färgkodade	x, y, z
Eller som enhetsvektorer	$\hat{e}_x, \hat{e}_y, \hat{e}_z$

Alla dessa går att generalisera till 3 och 4 dimensioner

3D: $\vec{a} + \vec{b} = (a_x + b_x,\ \ a_y + b_y,\ \ a_z + b_z)$

4D: $\vec{a} - \vec{b} = (a_x - b_x,\ a_y - b_y,\ a_z - b_z,\ \ a_w - b_w)$

TGA THE GAME ASSEMBLY

Lite mera vektormatematik

TGA
THE GAME ASSEMBLY

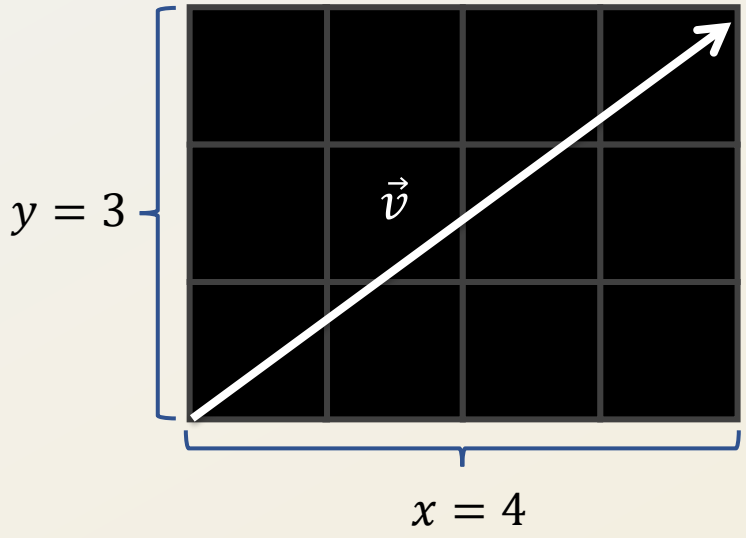
Normalisering av en vektor

Innebär att vi **skalar** den så att **längden** på **vektorn är 1**

Riktningen förblir oförändrad

$$\vec{v}_{norm} = \frac{\vec{v}}{\| \vec{v} \|}$$

Vi normaliserar genom att **dividerar vektorn** med sin längd



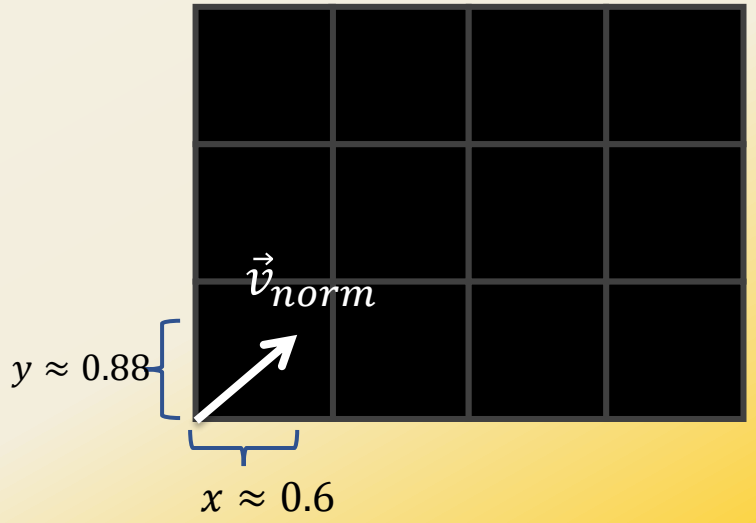
$$\|\vec{v}\| = \sqrt{4^2 + 3^2} = \sqrt{25} = \mathbf{5}$$

För att normalisera $\vec{v}$ dela med $\|\vec{v}\|$

$$\vec{v}_{norm} = \frac{\vec{v}}{\| \vec{v} \|} = \frac{[4 \quad 3]}{5} = \left[\frac{4}{5} \quad \frac{3}{5}\right]$$

$$\approx [\mathbf{0.8} \quad \mathbf{0.6}]$$

$$\|\vec{v}_{norm}\| = 1$$



TGA THE GAME ASSEMBLY

Normalicera noll-vektorn?

Nollvektorn kan inte normaliseras

- Nollvektorn: (0, 0)
- Har ingen riktning
- Längden är 0
- Normalisering innebär division med 0, vilket inte går!

TGA
THE GAME ASSEMBLY

Vektormultiplikation

Vi har sett **vektor** multiplicerat med **skalär**
Kan också multiplicera **vektorer med varandra**

Vi kommer titta på **tre sätt!**

- Skalärprodukt $\vec{u} \cdot \vec{v}$
  - **Resulterar** i en **skalär** (ett tal) och **inte** en ny **vektor**
  - Dot-product på engelska
  - Denna kommer vi **använda mycket**!

- Kryssprodukt $\vec{u} \times \vec{v}$
  - **Speciell** operation som **bara är meningsfull** i 3 dimensioner

- Komponentvis multiplikation $\vec{u} \otimes \vec{v}$
  - $\vec{u} \otimes \vec{v} = [u_x * v_x \quad u_y * v_y]$

TGA THE GAME ASSEMBLY

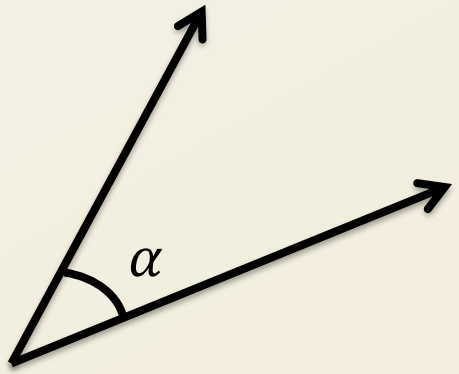
Skalärprodukt

Vi kan multiplicera **två vektorer** och få en skalär tillbaka – skalärprodukt (*dot product*)

2D: $\vec{a} \cdot \vec{b} = (a_x, a_y) \cdot (b_x, b_y) = a_x b_x + a_y b_y$

3D: $\vec{a} \cdot \vec{b} = (a_x, a_y, a_z) \cdot (b_x, b_y, b_z) = a_x b_x + a_y b_y + a_z b_z$

”Multiplicera komponentvis och summera produkterna”



Skalärprodukten går också att räkna ut på ett annat sätt:

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| * \left\|\vec{b}\right\| * \cos(\alpha)$$

α (”alfa”) är vinkeln mellan de två vektorerna

”multiplicera längden av vektorerna med cos för vinkeln mellan dem”

Kommutativ: $\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a}$

TGA THE GAME ASSEMBLY

Skalärprodukten $\vec{a} \cdot \vec{b} = \|\vec{a}\| * \|\vec{b}\| * \cos(\alpha)$

Vinkeln mellan två vektorer kan räknas ut om vi arrangerar om lite

Isolera $\cos(\alpha)$ genom att dela båda sidorna med $\|\vec{a}\| * \|\vec{b}\|$

$\frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|*\|\vec{b}\|} = \frac{\|\vec{a}\|*\|\vec{b}\|*\cos(\alpha)}{\|\vec{a}\|*\|\vec{b}\|}$ Stryker vi bort får vi kvar $\frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|*\|\vec{b}\|} = \cos(\alpha)$

Isolera α genom att ta inversen av cos på båda sidor

$$\cos^{-1}(cos(\alpha)) = \cos^{-1}\left(\frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|*\|\vec{b}\|}\right)$$

$$\alpha = \cos^{-1}\left(\frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|*\|\vec{b}\|}\right)$$

Om vi har **normaliserade** vektorer $\|\vec{a}\| = 1$, $\|\vec{b}\| = 1$ så är $\vec{a} \cdot \vec{b} = \cos(\alpha)$
Då kan vinkeln mellan vektorerna räknas ut genom $\boxed{\alpha = \cos^{-1}(\vec{a} \cdot \vec{b})}$

TGA THE GAME ASSEMBLY

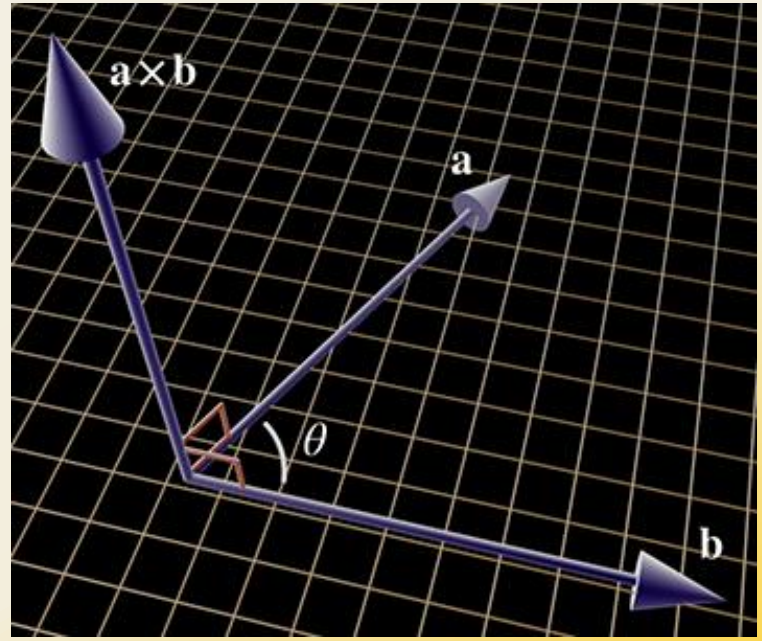
Kryssprodukt

att Vi kan multiplicera två **3D-vektorer** och få en vektor tillbaka – kryssprodukt (*cross product*)

Skrivs $\vec{a} \times \vec{b}$

Den resulterande vektorn är **ortogonal** (vinkelrät) mot **både** $\vec{a}$ **och** $\vec{b}$

Längden är $\|\vec{a}\|\|\vec{b}\| \sin(\alpha)$



TGA
THE GAME ASSEMBLY

Vektorprodukt | Kryssprodukt

Den är anti-kommutativ

Alltså: $\vec{a} \times \vec{b} = -(\vec{b} \times \vec{a})$

Den är inte associativ

Altså: $\vec{a} \times (\vec{b} \times \vec{c}) \neq (\vec{a} \times \vec{b}) \times \vec{c}$

$$\vec{a} \times \vec{b} = (a_y b_z - a_z b_y, \quad a_z b_x - a_x b_z \quad , a_x b_y - a_y b_x)$$

Min minnesregel, kanske hjälper det någon ^^

Att räkna x kräver yz, räka y kräver zx och z kräver xy
Alltid ab - ab

Tänk x, ***y, z, z, y****, x fram sen tillbaka*
Läs: ***x =*** *multiplicera* $\vec{a}_{\boldsymbol{y}}\vec{b}_{\boldsymbol{z}}$ *subtrahera* $\vec{a}_{\boldsymbol{z}}\vec{b}_{\boldsymbol{y}}$

Tänk y, ***z, x, x, z****, y fram sen tillbaka*
Läs: ***y*** *= multiplicera* $\vec{a}_{\boldsymbol{z}}\vec{b}_{\boldsymbol{x}}$ *subtrahera* $\vec{a}_{\boldsymbol{x}}\vec{b}_{\boldsymbol{z}}$

Tänk z, ***x, y, y, x****, z fram sen tillbaka*
Läs: ***z*** *= multiplicera* $\vec{a}_{\boldsymbol{x}}\vec{b}_{\boldsymbol{y}}$ *subtrahera* $\vec{a}_{\boldsymbol{y}}\vec{b}_{\boldsymbol{x}}$

TGA
THE GAME ASSEMBLY

AVSLUTNINGSVIS

TGA
THE GAME ASSEMBLY

Innan nästa föreläsning

Bonus:

- Ta in material från många källor!
  Många lite abstrakta koncept att förstå i den här kursen
  Bra att få dem förklarade på flera olika sätt
- Interaktiv linalg-bok: http://immersivemath.com/ila/index.html
- Bra youtubeserie om linalg: https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab

Räkneuppgift

- Övning 01 – Kom igång
- Övning 02 – Vektorer

Uppgift

- 01 – Vektorklasser

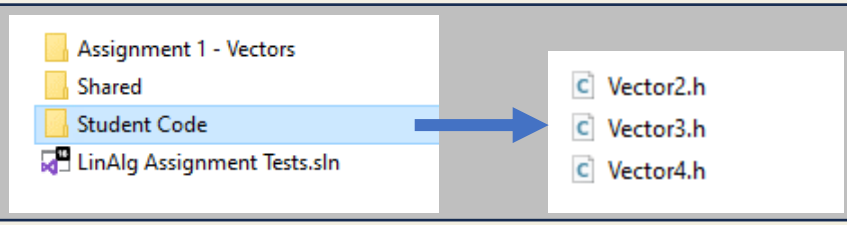
TGA THE GAME ASSEMBLY

Unit-test uppgifterna

Vektorklasser

Ladda ned projektet från omniway

- I Student code finns tomma interface
  - Det är dessa **ni** kommer att **implementera**



- Projektet innehåller **tomma** enhetstester
  - Utgå från dem och skriv egna tester
    - På **torsdag** får ni tillgång till våra enhetstester
    - Bilden till höger visar mycket av vilka tester vi skrivit
  - Gå in i menyn **Test** och öppna **Test explorer**
  - Kan behöva kompilera för att testerna skall synas

